



Ninety Thousand Acres (Offline) - Progress Log & Lessons Learned

This document tracks our development progress, serving as a step-by-step guide and reference for the project. It highlights key files, critical code concepts, common issues we've solved, and tips for working with Godot Engine.



Current Status

We have successfully completed **Phase 1 (Data Layer)** and **Phase 2 (Base Construction & Training)**.

- ☒ Economy System (Wood, Stone, Food)
 - ☒ Base Grid with Zoned Layout (UI Zone vs Grid Zone)
 - ☒ Building Placement (Main Tower, Barracks)
 - ☒ Unit Training System (30s Timer, Progress Bar)
 - ☒ Unit Spawning & Click Interaction
-



Phase 1: Project Setup & Data Layer

Goal: Establish the core data structures for resources, units, and buildings.



Files Created

1. `src/scripts/ResourceManager.gd`

- **Description:** Manages the global economy (Wood, Stone, Food).
- **Critical Code:**

```
signal resources_changed # Notifies UI when resources update
func spend_resource(type: String, amount: int) -> bool:
    # Checks balance and deducts if sufficient
```

- **Key Concept: Singleton Pattern.** This script is attached to a node in the scene tree, allowing other scripts to access it via `get_node("/root/BaseScene/ResourceManager")`.

2. `src/scripts/UnitStats.gd` & `BuildingStats.gd`

- **Description:** Data templates (Resources) for units and buildings.
- **Critical Code:**

```
extends Resource
class_name UnitStats # Makes this type globally accessible
@export var hp: int = 100 # Visible in Godot Inspector
```

- **Key Concept: Resources vs Nodes.** Resources are for data (stats, costs), while Nodes are for game objects (visuals, logic).

⚠ Common Issues & Fixes

- **Issue:** Identifier "UnitStats" not declared in the current scope.
 - **Cause:** Missing `class_name UnitStats` at the top of the script.
 - **Fix:** Always add `class_name YourClassName` if you want to reference the script type directly in other files.

💡 Tips & Tricks

- **Godot Inspector:** Use `@export` on variables in **Resource** scripts to edit values visually in the Editor without touching code.
- **Signals:** Use signals like `resources_changed` to decouple logic. The UI listens for changes rather than constantly checking the resource values.

🟦 Phase 2: Base Construction & UI

Goal: Create a visual base where players can place buildings and manage resources.

📁 Files Created/Updated

1. `src/scripts/BaseManager.gd`

- **Description:** The "brain" of the base. Handles grid logic, mouse clicks, and spawning.
- **Critical Code:**

```
const UI_HEIGHT = 200 # Separates UI area from Grid area
func _input(event):
    # Calculates grid coordinates based on mouse position
    var grid_y = floor((mouse_pos.y - UI_HEIGHT) / TILE_SIZE)
```

- **Key Concept: Zoned Layout.** By defining a `UI_HEIGHT`, we ensure clicks in the top panel don't accidentally place buildings, and visuals align perfectly with mouse clicks.

2. `src/scripts/BaseUI.gd`

- **Description:** Manages the HUD (Resource labels, Build buttons).
- **Critical Code:**

```
lbl_wood = find_child("LblWood", true, false) # Finds node by name recursively
btn_tower.pressed.connect(_on_tower_pressed) # Connects button signal
```

- **Key Concept: `find_child`.** A robust way to find UI elements without hardcoding long paths like `$Panel/MarginContainer/...`

3. `src/scripts/Building.gd` & `src/scripts/Unit.gd`

- **Description:** Visual representations. They draw themselves based on data passed from `BaseManager`.
- **Critical Code:**

```
func init(stats, pos, ui_offset_y):  
    global_position = Vector2(pos.x * 64 + 32, pos.y * 64 + 32 +  
    ui_offset_y)
```

- **Key Concept: Initialization Order.** We create the `ColorRect` (sprite) inside `init()` or `_ready()` to ensure it exists before we try to change its color.

4. `src/scenes/BaseScene.tscn`

- **Description:** The main scene structure.
- **Structure:**
 - `UI_Layer` (Top Panel for Resources/Buttons)
 - `GridBackground` (Dark gray area for buildings)
 - `BuildingsLayer` (Where buildings/units are added)

⚠ Common Issues & Fixes

- **Issue:** Buildings appearing white or colors not updating.
 - **Cause:** Trying to access `sprite.color` before the `sprite` node was created or added to the tree.
 - **Fix:** Ensure `sprite = ColorRect.new()` and `add_child(sprite)` happen *before* `_update_visuals()`.
- **Issue:** Clicks not registering on buildings.
 - **Cause:** `BaseManager._input()` was "eating" the click before the building could detect it, or `Area2D` was too complex.
 - **Fix:** Centralized all click logic in `BaseManager`. It checks if a building exists at the clicked tile coordinate.
- **Issue:** UI overlapping with the game grid.
 - **Cause:** No separation between UI space and game space.
 - **Fix:** Implemented `UI_HEIGHT` constant and offset all grid calculations and visual positions by 200 pixels.

💡 Tips & Tricks

- **Mouse Filter:** Set `mouse_filter = 1` (Ignore) on background `ColorRect` nodes so clicks pass through to the logic layer.
- **Debugging:** Use `print("Message")` extensively. We added debug prints to every major action (e.g., `"BaseManager: Placed Barracks"`) to trace the flow easily.

● Phase 3: Unit Production

Goal: Allow players to train units from the Barracks with a real-time timer.

📁 Files Created/Updated

1. `src/scripts/TrainingUI.gd`

- **Description:** The popup panel for training units.
- **Critical Code:**

```
func _process(delta):  
    if is_training:  
        timer -= delta  
        progress_bar.value = (1.0 - (timer / training_time)) * 100.0
```

- **Key Concept: Delta Time.** Using `delta` ensures the timer runs smoothly regardless of frame rate.

2. `src/scripts/UnitPool.gd`

- **Description:** Stores trained units and emits a signal when a new one is ready.
- **Critical Code:**

```
signal unit_added(unit_stats)  
func add_unit(unit):  
    trained_units.append(unit)  
    unit_added.emit(unit)
```

⚠️ Common Issues & Fixes

- **Issue:** `Invalid access to property... on a base object of type 'null instance'.`
 - **Cause:** The script tried to access a button (e.g., `btn_train.pressed`) before the scene was fully loaded.
 - **Fix:** Use `@onready var btn_train = $Path` or check `if btn_train != null` before connecting signals.
- **Issue:** Training UI not appearing when clicking Barracks.
 - **Cause:** The signal connection between `BaseManager` and `TrainingUI` was missing or the node path was wrong.
 - **Fix:** Verified node paths in `BaseScene.tscn` and used `get_node_or_null` with clear error messages.

💡 Tips & Tricks

- **Progress Bars:** Godot's `ProgressBar` node is perfect for timers. Just update its `.value` property (0.0 to 100.0).
- **Auto-Close UI:** Use `await get_tree().create_timer(1.5).timeout` to pause execution and auto-close panels after an action completes.

General Development Tips

1. Godot Editor Features

- **Remote Scene Tree:** While the game is running, click the "Remote" tab in the Scene dock. You can inspect live nodes, see their current properties, and debug layout issues in real-time.
- **Debugger:** The "Debugger" tab at the bottom shows errors and warnings. Always check this first if something isn't working.

2. GDScript Best Practices

- **Type Hints:** Use `var health: int = 100` instead of `var health = 100`. It helps catch errors early.
- **Constants:** Use `const TILE_SIZE = 64` for magic numbers. If you change the tile size later, you only change it in one place.
- **Naming Conventions:**
 - Scripts: `PascalCase` (e.g., `BaseManager.gd`)
 - Variables/Functions: `snake_case` (e.g., `unit_pool`, `spawn_unit()`)
 - Constants: `UPPER_SNAKE_CASE` (e.g., `UI_HEIGHT`)

3. Scene Management

- **Instancing:** Use `preload("res://path/to/scene.tscn").instantiate()` to create objects dynamically.
 - **Groups:** Assign nodes to groups (e.g., `groups = ["base_manager"]`) to find them easily using `get_tree().get_first_node_in_group()`.
-

Next Steps

We are ready to move to **Phase 4: World Map & Troop Movement**.

- Implement the 100x100 random map generation.
- Create the camera system for panning/zooming.
- Develop the troop movement logic (real-time travel between tiles).

Keep this document updated as we progress!